

CLAUDE.md Hierarchy

Teach Claude your project once — not every session.

IN THIS LECTURE,

1 What CLAUDE.md is

2 The hierarchy

3 How levels combine

4 Nested (directory) memory

5 @import & /memory

6 What belongs in it

What Is CLAUDE.md?

A standing note Claude reads before it starts work.

Persistent project memory, loaded automatically

- ▶ Read every time Claude Code starts a session
- ▶ Write your conventions and gotchas once
- ▶ Stop re-explaining your project each time



? Anything you'd otherwise re-explain every session — build commands, conventions, gotchas — lives here once.

The Hierarchy

Company policy, team rules, your desk.

user

~/.claude/CLAUDE.md · your
prefs, all projects



project

./CLAUDE.md · team,
committed to git



directory

nested CLAUDE.md · per-
folder rules



 Know where each lives: user = global prefs, project = shared & version-controlled, directory = per-folder. (Enterprise policy sits on top.)

How the Levels Combine

They stack together — the most specific wins.



Combined into context, not one replacing another

- ▶ All levels merge into one working context
- ▶ On a conflict, the more specific / later level wins
- ▶ Project beats user; the closer rule takes priority



Personal file says 2-space, project says 4-space? The project wins — the closer, more specific rule takes priority.

Nested (Directory) Memory

Folder rules load only when Claude works there.

A CLAUDE.md inside a folder loads on demand

- ▶ frontend/CLAUDE.md kicks in only for frontend files
- ▶ Perfect for large monorepos
- ▶ Each area gets the right rules, ignores the rest



A repo with 50 sub-projects won't drown Claude in irrelevant rules — nested files load only where they apply.

@import & /memory

Keep it modular, and edit it easily.



Split it up, and manage it simply

- ▶ @import → pull in other files (@docs/style.md)
- ▶ /memory → open memory files to edit
- ▶ Type # to quick-add a rule — Claude asks which file

💡 Imports keep the file organised — but everything still loads at launch, so it doesn't shrink your context.

📌 Type # at the start of a message to jot a rule; Claude asks which file to save it to.

What Belongs in CLAUDE.md

Write what Claude can't guess — skip what it can.



Keep

- ✓ Pitfalls & conventions that differ from defaults
- ✓ The why behind a rule



Cut

- ✗ Things Claude can figure out itself
- ✗ Folder layout, dependency lists



Keep it under ~200 lines — a bloated CLAUDE.md actually reduces adherence. Lean, high-signal files work best.

Key Takeaways

CLAUDE.md, in three lines.

1

Auto-loaded memory

CLAUDE.md is project memory
Claude reads automatically — write it once.

2

Levels stack

user → project → directory combine;
the more specific rule wins.

3

Modular & lean

Use @import to stay modular,
/memory to edit; keep it rule-focused.



End of Lecture 3.1 — CLAUDE.md Hierarchy

Slash Commands & Skills

Save a workflow once, run it with one word.

IN THIS LECTURE,

1 The repetition problem

2 Custom slash commands

3 Command frontmatter

4 Skills

5 Skill frontmatter that matters

6 Commands/skills vs CLAUDE.md

The Repetition Problem

Stop retyping the same long instruction.

Save it once as /review

- ▶ Typing the same paragraph every time drifts
- ▶ One day you forget a step
- ▶ Capture it once, run it identically



A saved command is like speed-dial — the same precise instruction, one short name, every time.

Custom Slash Commands

A Markdown file becomes a command.



Drop a file in `.claude/commands/`

- ▶ Project scope (`.claude/commands/`) or personal (`~/.claude/commands/`)
- ▶ Filename = command · body = the prompt
- ▶ `$ARGUMENTS` drops your input into the prompt

Example: `fix-issue.md` → `/fix-issue 123` (`$ARGUMENTS = '123'`)



Project scope is committed, so the whole team gets the command; `~/.claude/` is just you.

Command Frontmatter

A few lines of config make it robust.



description

Shows in /help; lets Claude auto-suggest it



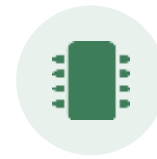
argument-hint

Autocomplete hint for the input



allowed-tools

Pre-approve the tools it needs



model

Pin a cheaper, faster model



description also lets Claude offer the command automatically when your request matches it.

Skills: the Richer Form

Commands grew up into skills.

Commands are now merged into skills

- ▶ `.claude/skills/<name>/SKILL.md` also makes `/name`
- ▶ Adds a folder for supporting files
- ▶ Auto-discovery — Claude can invoke it when relevant



🔗 Same `/command`, more power: skills carry extra files and can be invoked by Claude, not just by you. On a name clash, the skill wins.

Skill Frontmatter That Matters

Three fields the exam cares about.



context: fork · allowed-tools · disable-model-invocation

- ▶ context: fork → run in an isolated context
- ▶ allowed-tools → pre-approves (does NOT restrict)
- ▶ disable-model-invocation: true → only you can trigger it



allowed-tools pre-approves the listed tools — it does NOT block the others. Every other tool stays available.



Use disable-model-invocation for side-effect commands like /commit or /deploy, so Claude never fires them on its own.

Commands/Skills vs CLAUDE.md

Always-on rules, or on-demand actions?



CLAUDE.md

- ✓ Passive context Claude always reads
- ✓ Standing conventions

VS



Command / Skill

- ✓ An action you (or Claude) invoke
- ✓ Repeatable tasks, on demand



Standing convention → CLAUDE.md; repeatable task → a command/skill. Both come in project & personal scope.

Key Takeaways

Commands & skills, in three lines.

1

File → /command

A Markdown file in
.claude/commands/ becomes a
/command; \$ARGUMENTS passes
input.

2

Skills add power

SKILL.md adds supporting files, auto-
discovery, and context: fork.

3

On-demand vs always-on

allowed-tools pre-approves (not
restricts); CLAUDE.md = always-on,
commands = on-demand.



End of Lecture 3.2 — Slash Commands & Skills

Path-Specific Rules

The right rules, only where they apply.

IN THIS LECTURE,

1 Why one CLAUDE.md is too broad

2 .claude/rules/

3 Scoping with paths:

4 Conditional loading

5 Rules vs subdirectory CLAUDE.md

6 Precedence

Why One CLAUDE.md Is Too Broad

Backend rules shouldn't clutter frontend work.

A single file applies to everything

- ▶ In a large repo, most of it is irrelevant to any one task
- ▶ API-security rules don't help when editing CSS
- ▶ You want rules that appear only in their area



? You want zone-specific signage — pool rules at the pool, gym rules at the gym — not one sign for the whole building.

.claude/rules/

One folder, many focused rule files.



Each .md loads as project memory by default

- ▶ Drop focused files: api.md, tests.md, style.md
- ▶ Same behaviour as CLAUDE.md, just split up
- ▶ A tidier way to organise your project rules

📌 **.claude/rules/*.md and .claude/CLAUDE.md sit at the SAME priority — rules are just a tidier way to split them up.**

Scoping With paths:

Add a glob, and the rule wakes up only for matching files.

A paths: frontmatter scopes the rule

- ▶ paths: is a list of glob patterns
- ▶ The rule loads ONLY when Claude touches a match
- ▶ e.g. paths: ['src/api/**']



The paths: glob is the trigger — 'src/api/**' means apply this rule only inside the API folder.

Conditional Loading

With paths = on demand. Without = always.



With paths:

- ✓ Loads only for matching files
- ✓ Conditional — area-specific



No paths:

- ✓ Loads every session
- ✓ Unconditional — project-wide



Mix both: always-on rules for project-wide conventions, path-scoped rules for area-specific ones.

Rules vs Subdirectory CLAUDE.md

Two ways to scope by location — pick the right one.



Glob rule (paths:)

- ✓ Flexible, cross-cutting patterns
- ✓ e.g. every *.test.js anywhere

VS



Subdirectory CLAUDE.md

- ✓ Everything under one folder
- ✓ A whole subtree of the repo

🔗 Exam distinction: a pattern that spans the repo → glob rule; everything in one directory → a CLAUDE.md in that directory.

Precedence

Path rules land last — and last usually wins.

user → project → path-specific (appended last)

- ▶ Path rules append when you touch matching files
- ▶ On conflict, the later / more specific rule wins
- ▶ Consistent with the CLAUDE.md hierarchy



🔗 Same principle as 3.1 — broader first, most specific last, and the closest rule wins.

Key Takeaways

Path rules, in three lines.

1

Rules folder

.claude/rules/*.md load as project memory; a paths: glob scopes a rule to matching files.

2

Conditional vs always

With paths: = conditional (on demand); without = always-on.

3

Glob vs folder

Glob rule for cross-cutting patterns; subdirectory CLAUDE.md for a whole folder.



End of Lecture 3.3 — Path-Specific Rules

Plan Mode vs Direct Execution

Measure twice, cut once.

IN THIS LECTURE,

1 Two ways to work

2 What plan mode is

3 How to trigger it

4 When to use it

5 The Explore subagent

6 The plan loop

Two Ways to Work

Quick errand, or a project that needs a plan.



Direct

- ✓ Small, obvious change
- ✓ Just do it

VS



Plan mode

- ✓ Big or risky change
- ✓ Think first, then act

🔍 Like a carpenter: measure twice, cut once. Cheap changes don't need a plan; expensive mistakes do.

What Plan Mode Is

Read-only. Claude plans; it can't change anything yet.

Explore and write a plan — no edits until you approve

- ▶ Claude reads, searches, drafts a numbered plan
- ▶ Write and Edit are blocked at the tool level
- ▶ You review every change before it's real



It's not a polite request — plan mode BLOCKS file-modifying tools until you say go.

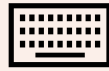
How to Trigger It

Three ways in.



Shift+Tab ×2

Cycles Default → Auto-Accept → Plan



/plan

Type it in the prompt



--permission-mode plan

Start the session in plan mode



Watch the status line — it shows 'plan mode on'. The **--permission-mode plan** flag also works with headless **-p** runs.

When to Use It

The bigger the blast radius, the more you plan.



Plan for risk; skip it for quick edits

- ▶ Plan: multi-file, refactors, migrations, auth/payment/production
- ▶ Plan: unfamiliar repos, ambiguous tasks
- ▶ Skip: single-file tweaks, quick edits

💡 Small decisions compound: 20 unguided choices at 80% each $\approx$ 1% right. A plan catches the wrong turns before they cost you.

The Explore Subagent

Send a scout ahead — keep your desk clean.

A read-only scout that reports back a summary

- ▶ Read/Grep/Glob only — it investigates, doesn't modify
- ▶ Works in a separate context window
- ▶ Only the findings return to your main session

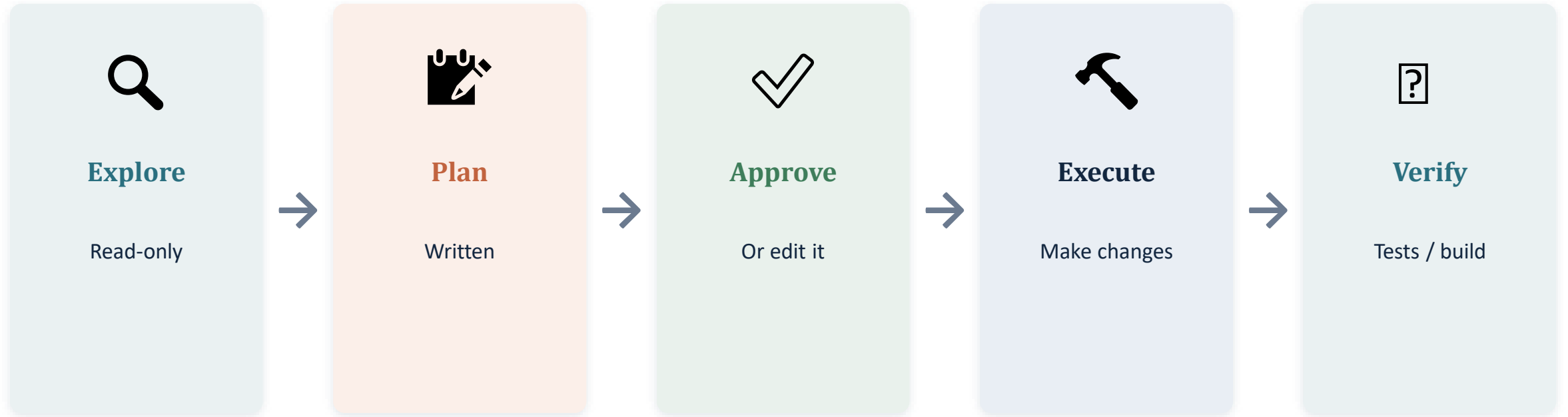


🔗 Same isolation idea as the subagents in 1.2–1.3 — exploration happens elsewhere, only findings come back.

🎯 This keeps a huge codebase's file-reads out of your main context so planning stays focused.

The Plan Loop

Explore → Plan → Approve → Execute → Verify.



You can edit the plan before approving — fixing a plan is fast because nothing's been built yet.

Key Takeaways

Plan mode, in three lines.

1

Read-only planning

Plan mode = Claude plans, you approve, then it edits — no changes until then.

2

Trigger for risk

Shift+Tab×2, /plan, or --permission-mode plan; use it for big/risky work.

3

Scout with Explore

The Explore subagent investigates in a separate context and reports back.



End of Lecture 3.4 — Plan Mode vs Direct Execution

Iterative Refinement

Great results come from passes, not one perfect shot.

IN THIS LECTURE,

1 The iterate mindset

2 Input/output examples

3 The interview pattern

4 Test-driven iteration

5 Sequential vs parallel fixes

The Iterate Mindset

Treat the first output as a draft.

Don't one-shot — refine in passes

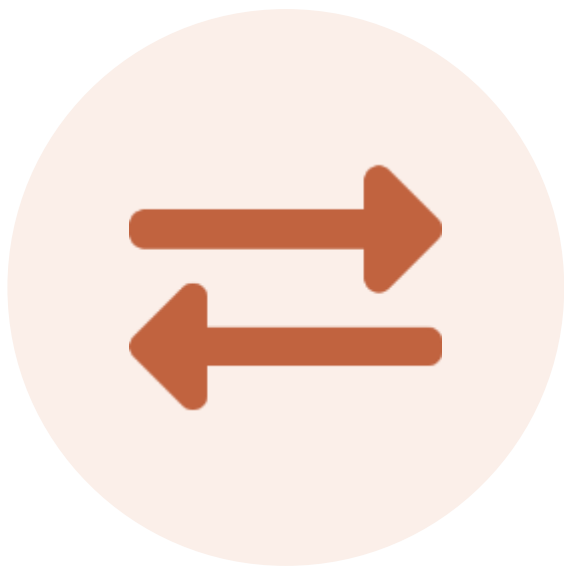
- ▶ The first result is a starting point, not the finish
- ▶ Refine it against something real
- ▶ Aiming for a flawless one-shot usually wastes time



Like a tailor's trial fittings — you adjust against something real, not guess the perfect cut upfront.

Steer With Input/Output Examples

Show the shape you want.



Give a sample input and the exact output

- ▶ Examples pin down format and detail
- ▶ They remove guesswork better than more description
- ▶ Show, don't just tell

↪ This is worked examples used as a steering move here — the full few-shot technique is taught in Domain 4 (4.2).

The Interview Pattern

Let Claude ask you the questions.

Ask Claude to interview you first

- ▶ It surfaces decisions you hadn't pinned down
- ▶ Which framework? How to handle errors? Edge cases?
- ▶ A fuzzy request becomes a precise spec



The questions expose gaps before Claude builds the wrong thing — a vague ask becomes a precise spec.

Test-Driven Iteration

Write the test first, iterate to green.



Define the test that must pass

- ▶ Then let Claude iterate until it does
- ▶ The test is an objective, unambiguous target
- ▶ No arguing whether the code is right — it passes or it doesn't



A passing test is an unambiguous goal — Claude knows exactly when it's done, and so do you.

Sequential vs Parallel Fixes

Order your fixes by whether they depend on each other.



Independent → parallel

- ✓ Unrelated fixes in one go
- ✓ Saves round-trips



Dependent → sequence

- ✓ One fix builds on another
- ✓ Order matters — avoids rework



Batching independent fixes saves round-trips; forcing dependent fixes in the right order avoids rework.

Key Takeaways

Iterative refinement, in three lines.

1

Iterate, don't one-shot

Treat the first output as a draft — refine in passes.

2

Steer & interview

Use input/output examples, and let Claude interview you to sharpen the ask.

3

Test & order

Iterate to a passing test; batch independent fixes, sequence dependent ones.



End of Lecture 3.5 — Iterative Refinement

Claude Code in CI/CD

Put Claude on the assembly line.

IN THIS LECTURE,

1 Headless mode

2 The -p flag

3 Structured output

4 Safety guards

5 CLAUDE.md as CI context

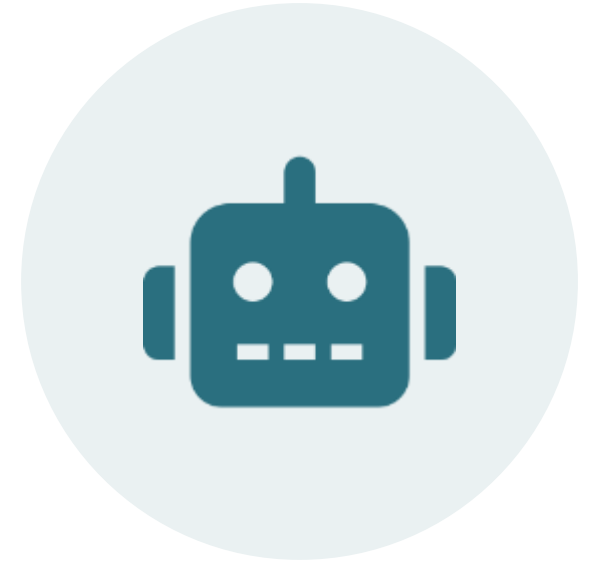
6 Self-review isolation

Headless Mode

No keyboard, no terminal — just a pipeline step.

Claude Code runs unattended inside CI/CD

- ▶ On every pull request, on a schedule, in a script
- ▶ No interactive terminal, no human clicking approve
- ▶ Just another automated stage



Like an automated night shift — the same Claude, running your task without anyone watching.

The -p Flag

One prompt in, one result out, then exit.



claude -p 'your prompt'

- ▶ Runs to completion, prints the result, exits
- ▶ Exits with a status code your pipeline can branch on
- ▶ No interactive session — no REPL



-p (print mode) is THE headless switch — the foundation of every CI step, cron job, and GitHub Action.

Structured Output

JSON your scripts can actually read.

--output-format json + --json-schema

- ▶ --output-format json → structured result + metadata (parse with jq)
- ▶ --json-schema → force an exact shape into structured_output
- ▶ Formats: text (default), json, stream-json (real-time)



💡 A downstream step branches on the result? Use --json-schema so the shape is guaranteed, not hoped for.

📌 json returns result + metadata like cost and session_id; parse it with jq.

Safety Guards

Unattended means you set the limits up front.



Cap the run before you walk away

- ▶ `--max-turns` (cap work) · `--allowedTools` (narrow permissions)
- ▶ `--max-budget-usd` (cap cost)
- ▶ Check the output content, not just the exit code



No human is watching — without turn and budget caps, a stuck run can burn time and money quietly at 3 a.m.

CLAUDE.md as CI Context

Your project rules ride along into the pipeline.

The same CLAUDE.md loads in CI

- ▶ Your conventions become the reviewer's context — automatically
- ▶ Write your rules once; CI inherits them
- ▶ No extra setup to teach the pipeline your standards



🔗 This is Lecture 3.1's CLAUDE.md doing double duty — write your rules once, and CI inherits them for free.

Self-Review Isolation

A fresh reviewer, not the author grading its own homework.



Run the reviewer as a separate instance

- ▶ Independent context — didn't write the code
- ▶ No bias from justifying its own choices
- ▶ An honest second opinion



A separate instance gives an unbiased second opinion — not the same session justifying its own choices.



The full multi-instance review architecture is taught in Domain 4 (4.6) — here it's the CI mechanic.

Key Takeaways

Claude Code in CI/CD, in three lines.

1

-p is headless

claude -p runs unattended — prompt in, result out, exit code.

2

Shape + guard

--output-format json + --json-schema
give scripts a reliable shape; cap
turns, tools, and budget.

3

Context + isolation

CLAUDE.md loads in CI; run self-
review as a separate, unbiased
instance.



End of Lecture 3.6 — Claude Code in CI/CD